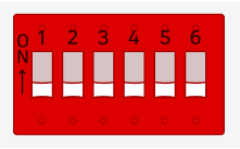
Garage Door Opener

For this project, you'll use a **few types of Switches** as well as a **Liquid Crystal Display** to create a *fortune generating machine.* The concept of pull-up resistors in the context of reading switch states will be reinforced as well.

Materials:

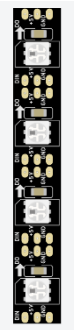


1 x Dip Switch (6)

Resistor:
2 x 220Ω

1 x Servo

1 x Pushbutton

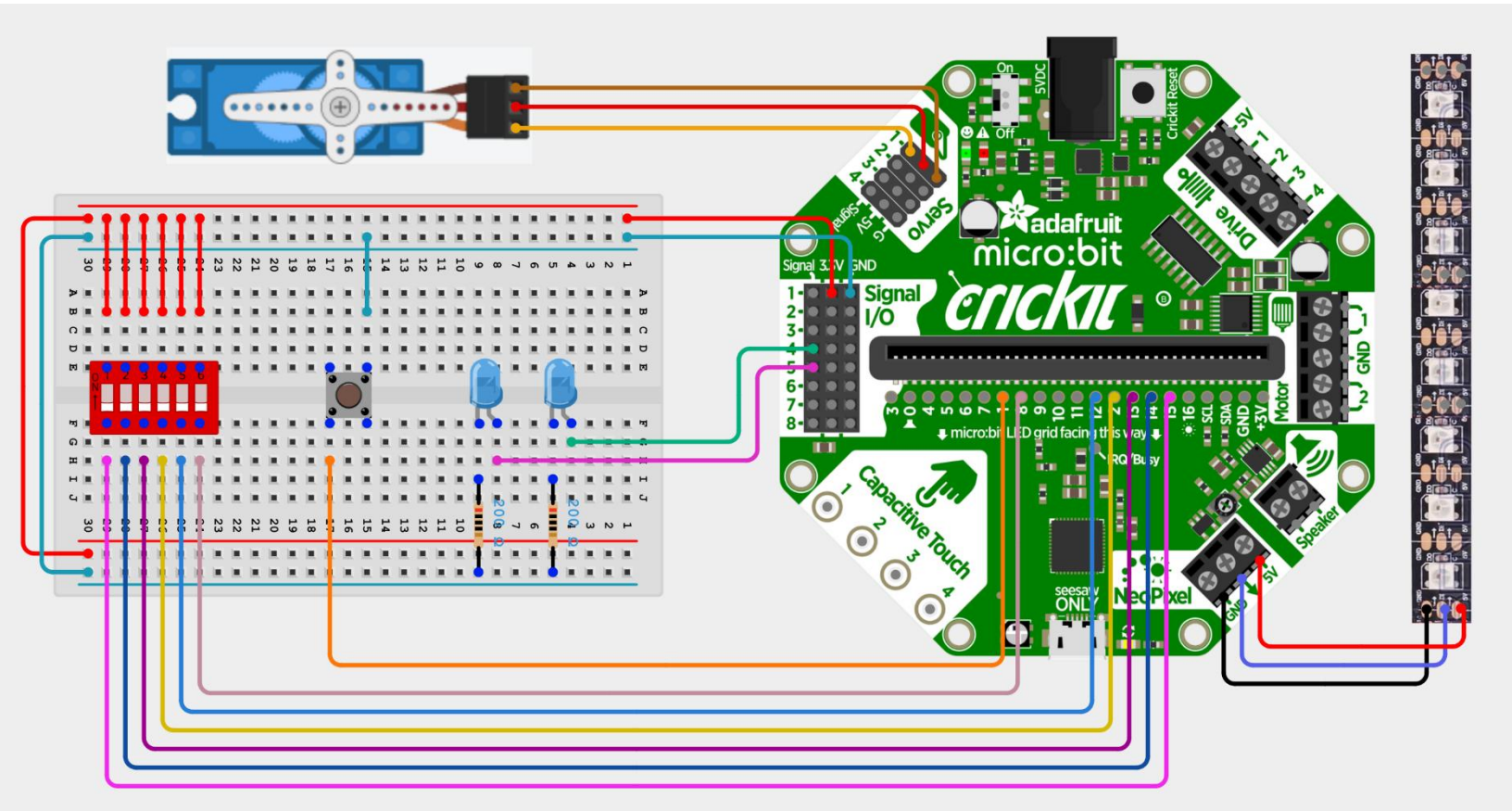


1 x Neopixel Strip

2 x LED (blue)

Wiring Diagram

Wire your project according to the following diagram.



Verify Circuit

As the complexity of this circuit is increasing, so is the possibility of errors in component placement (or poor connections for the physical circuit). Before getting into the complexity of the whole project, it would be a good idea to **verify that you've wired the components correctly**, by individually running a simple test on each.

Note: *Once you're done a test, disable that code and start on the next test. You shouldn't need to run all the tests concurrently.*

Test One: Servo Motor

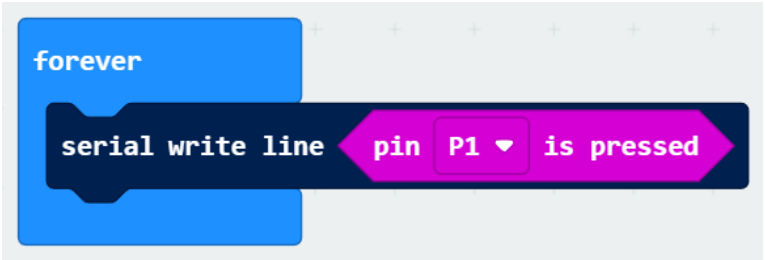
To ensure your servo is wired correctly and in good repair, have it move across its range of motion. Remember to include a wait to allow the Servo to reach its target angle.

- **NOTE:** This example code specifies **servo angle** by pulse width rather than degrees. The main reason is a "bug" where servo motion is *roughly half of what it should be* for most servos when using the **degree blocks.**
    - This is because the servo angle block maps to a particular PWM range (1000 – 2000, likely), but not all servos use that as their full range.

.

Test Two: Pushbutton

Since the pushbutton is connected to **pin 1,** you can use the blocks from the Input palette.

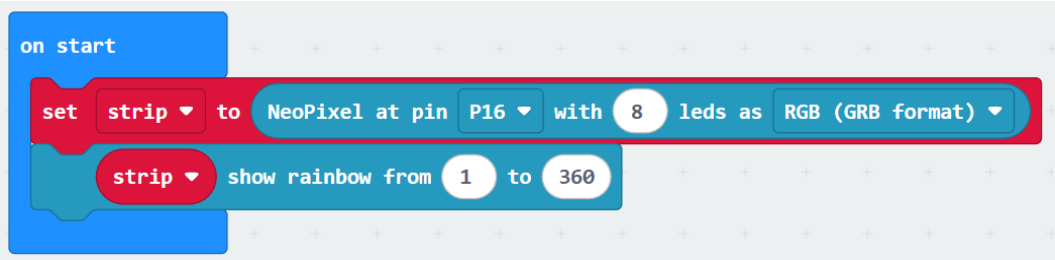


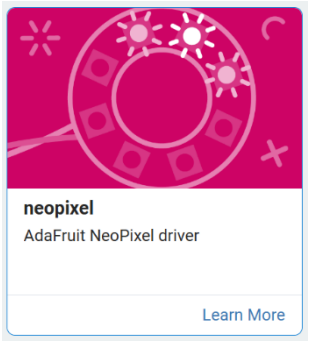
Note: While using the pink blocks is very simple, be aware that it will only work for buttons wired to pins 1 and 2.

Pin 0 can read capacitive touch, but not mechanical hardware buttons.

Test Three: Neopixel

A few of the built-in **fill** blocks allow this component to be tested easily.

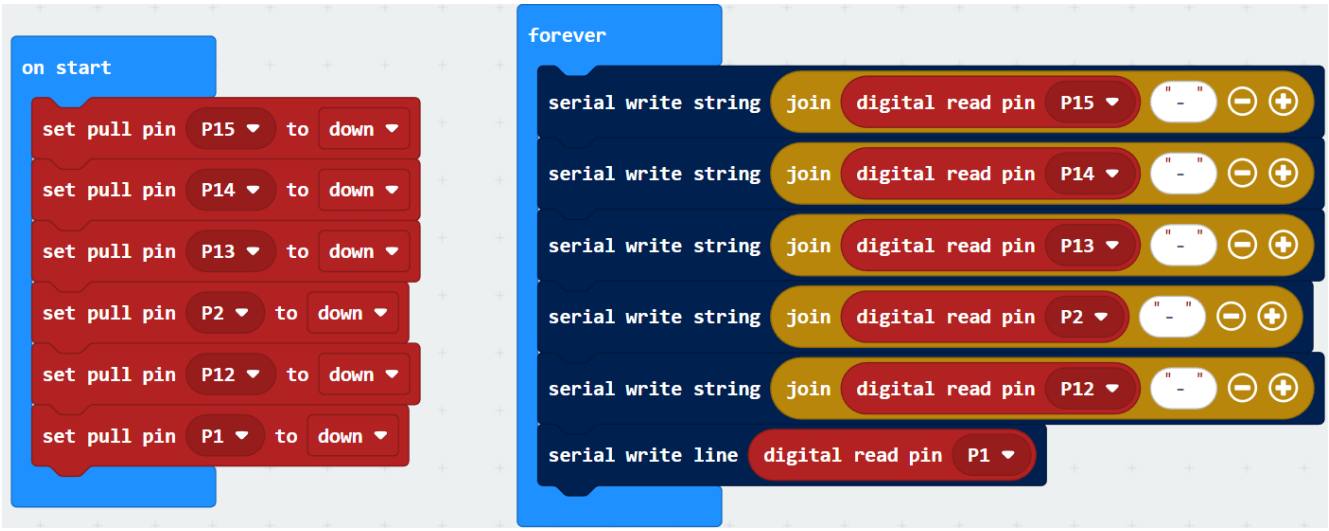




Don't forget to add the ***Neopixel Extension***

Test Four: DIP Switch

Although it looks like **one component,** you must test each individual switch. Since they are connected to the micro:bit pins directly, the onboard pull-down resistors can be enabled.



NOTE the difference between **write string** and **write line.**

write string: prints to the terminal, but doesn't advance the cursor to the next line

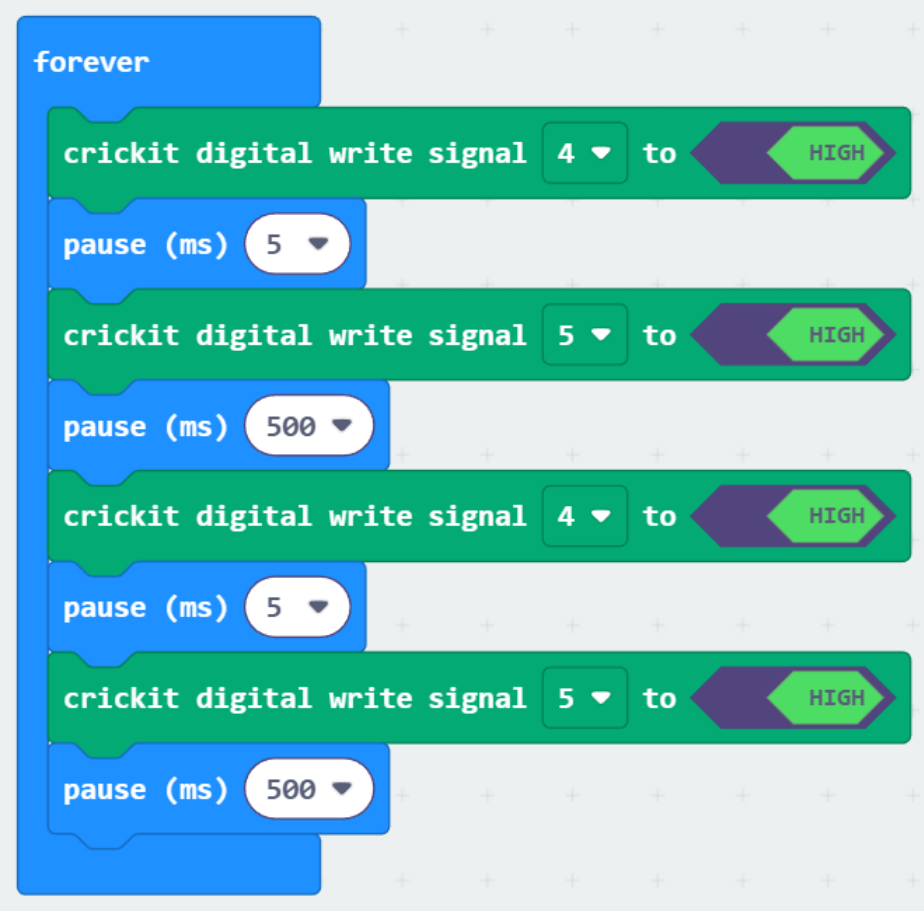
write line: prints to the terminal, and advances the cursor to the next line

This should result in printing out the **switch states** all on the same line:

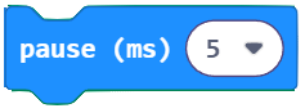
- 1-0-0-1-0-1 //1 → switch off 0 → switch on

Test Five: LEDS

Using Crickit Signal Pins, cycle them on/off…



Note: You might wonder why there are some very small delays in this test code…



Any time we are reading or writing with the **Crickit signal pins**, we need to be cautious of too many instructions in quick succession.

- Otherwise, the Crickit board may stall (2-10 seconds unresponsive)
- Adding even a very short delay like this should be enough

Assignment Details: Base Features

For this assignment, completing the following base features will earn you a maximum mark of 8.5/10. Some hints and explanation will be given, but before looking at those things you should try to construct a solution on your own.

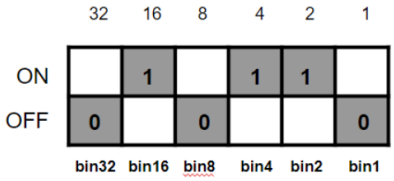
The remaining assignment grade 1.5/10 can be earned by completing some of the extensions *(information further on in document)*

Binary Code:

As we saw, an early strategy for password protected electronic signals was to use a hardware dipswitch to define the code required to activate a garage door opener.

Your first task should be to:

- Read all 6 dip switch positions
- Treat each as a single bit in a binary number
  - Add up the decimal value of each place which holds a **1** and store this value in a variable called **dipValue.**



This example should result in **22** being stored in **dipValue.** *(16 + 4 +2)*

Visualize the Code (Neopixel)

Use the neopixel to visualize which dip switches are **on** and which are **off:**

- Only need to use the first 6 LEDs for this feature
- If the current dipswitch pattern isn't correct:
  - Turn one Neopixel on *for each dip switch* that is in the **on state**
  - The position of these LEDs should match the positions on the LED strip.
  - Have the enabled LEDs all be the same color, **but not green**
- If the dipswitch pattern is correct:
  - Continue to visualize the dip switch positions but **use the color green.**

Open/Close the Door:

Make sure you have a **doorCode variable** that holds the target secret code. Please use **43** as the target sample code (101011).

Create a function called **checkCode(),** which should be called whenever the push button is activated. The function should compare **doorCode** and **dipValue.**

If they are equal, do one of the following:

OPEN THE DOOR *(if it is currently closed)*

- Print "Door Opening" to the Serial Monitor
- Have the servo move to 150 degrees
- Wait 1 second *(servo travel time)*
    - During this time, have the **TWO BLUE LEDS alternate on/off**

CLOSE THE DOOR *(if it is currently open)*

- Turn off LEDs and print "Door Closing" to the Serial Monitor
- Have the servo move to 30 degrees
- Wait 1 second *(servo travel time)*
    - During this time, have the **TWO BLUE LEDS alternate on/off**

If doorCode and dipValue are not equal:

- print "Incorrect Code" to the Serial Monitor

Extensions

The remaining assignment grade 1.5/10 can be earned by completing **at least two** of the following extensions.

Emergency Manual Mode

Holding the pushbutton for **2 seconds** should enter "manual mode."
In manual mode:

- Pressing pushbutton should open or close immediately *(skips checking dipswitch pattern)*
- LEDs turn purple (or any unique color) to indicate manual control
- Code unlocking is disabled until manual mode is exited (another long-press to disable)

Audio Feedback System

Use the Micro:bit speaker *(or a larger one wired to the Crickit board)* to create sound effects for system events:

- Short melody → correct code *(on button press)*
- Low buzz → incorrect code *(on button press)*
- Rising tone → opening
- Falling tone → closing

Obstruction Safety Sensor

Simulate a garage-door safety system (like the real infrared beam).
Use either a photoresistor or ultrasonic sensor to act as an **obstruction sensor**.
If the sensor is triggered while the door is closing:

- Stop the servo immediately
- Flash the blue LEDs
- Print **"Obstruction Detected"** to the Serial Monitor
- Re-open the door

For this feature, you'll have to figure out what available pins can be used for your "sensor".

It might be useful to look back at past assignments and circuit diagrams…

Attempt Counter and Lockout Mode:

Add a security feature that counts incorrect attempts.
If the user tries the pushbutton with the wrong DIP code **three times in a row**, the system should:

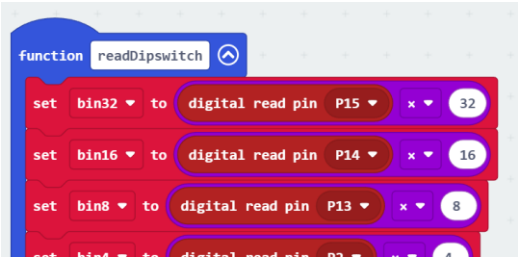
- Print **"System Locked"** to the Serial Monitor
- Disable the pushbutton for a set amount of time (5–20 seconds)
- Flash the neopixels **red** during the lockout period

------ Suggestions and Hints are below but try to build what you can on your own first! ------

Hints and Suggestions:

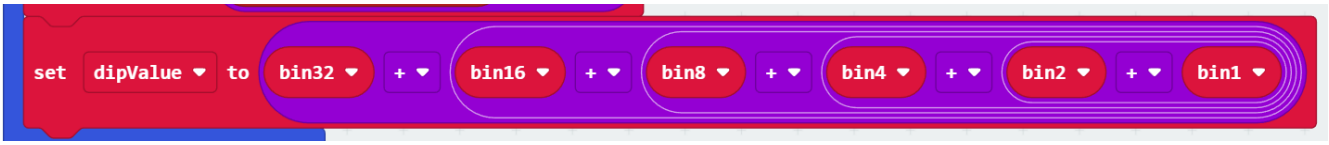
Binary Code:

Aim to create and use **Helper Functions** whenever possible. For this feature, a function called **readDipswitch** would be a good place to start. Also, create one variable for each of the dipswitch readings.



And so on…

This *ALMOST* completes the task. What remains is to **sum up** all these variables to get the current dipswitch code value.



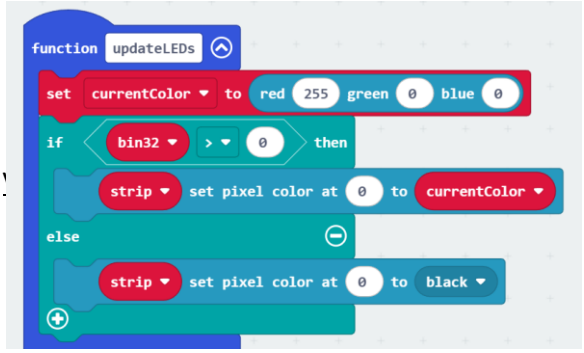
Visualize the Code:

For this feature, you should already have some data related to the dipwitches stored in the individual variables.

If you've followed the suggestions above *(not the only way to solve the problem!), each variable will either hold:*

0 → dipswitch at that position was off ***value > 0 → dipswitch at that position was on***

Knowing this, you just need 6 IF/ELSE Statements which check each variable and control one Neopixel LED based off what you see.

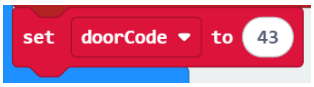


- Instead of hardcoding the LED color in, use a variable. This will be more flexible as some features / extensions want the Neopixel to use different colors.
- Recall that setting an LED to **black** is the same as turning it off.

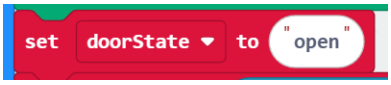
Repeat this idea for the remaining 5 variables / LEDs

Open the Door

First off, you should ensure that in **setup** you've created a variable to hold the secret code:

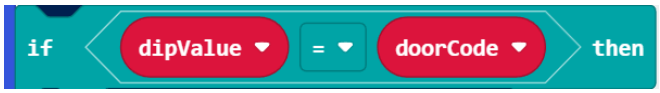


Second, we need a way to keep track of if the door is currently **open** or **closed.** A string-type variable makes for a fairly clear way to do this:

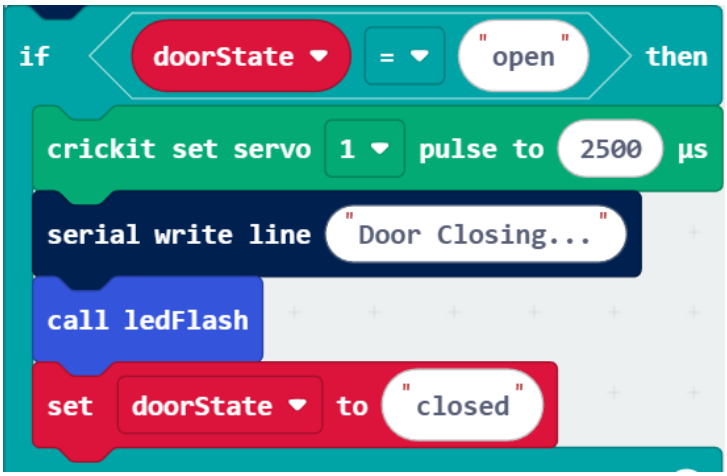


Don't forget to grab  *from the **text** palette*

Then, whenever the pushbutton is pressed, check if the **doorCode** and your calculated **dipValue** are the same amount:



If they are equal, the code should be accepted and the door should either open or close, depending on what its current state is. Here's an example of what would be done if it were currently open:

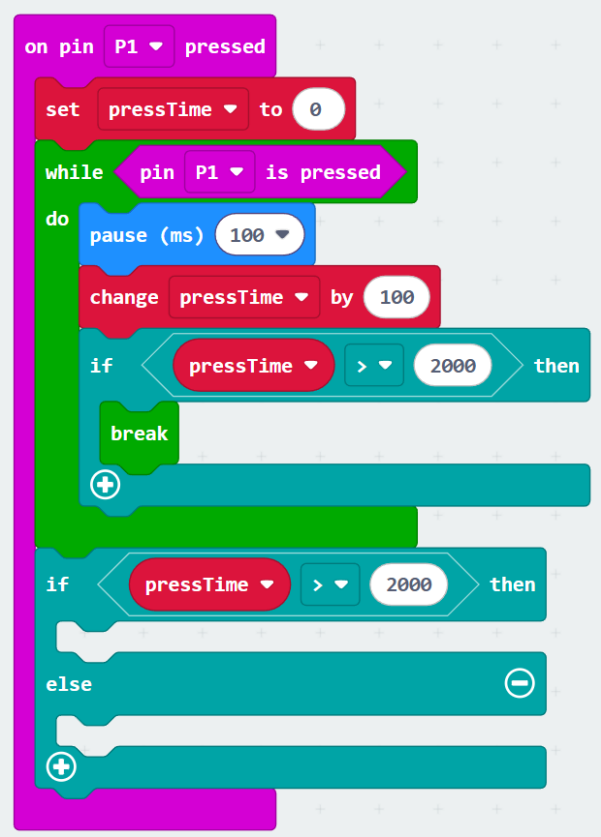


Implement a similar idea for moving the other direction in the **else** case…

Emergency Manual Mode

One of the simplest ways to build this feature is to use a **loop to add to a variable** once every tenth of a second, **for as long as the button is being held down.**

When the button is released (or the counting variable reaches at least 2000), the loop will end. At that point it should be possible to check the **counting variable's value** to determine if the button got a **short press** or **long press.**



To handle the rest of this extension, make a variable (something like **manualMode**) and set it initially to **false.**

- On a long press have it flip its value (true becomes false, false becomes true).
- Add some **if/else** logic into the code that currently handles the opening of the door
  - It should check your **manualMode** variable, and if it is **true** the door should be activated regardless of the current dipswitch code.

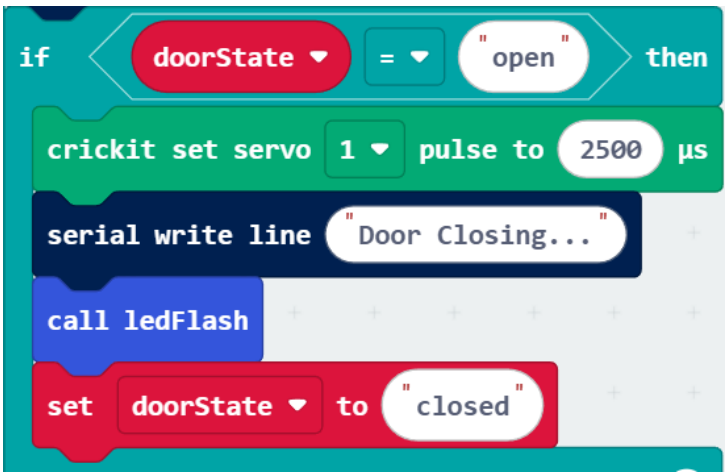
Audio Feedback System

No hints for this one – just be creative! 🎵 🎶 🎵

Obstruction Safety Sensor

The **first priority** for this feature should be to correctly wire a sensor and be able to get reliable readings. How to do so depends on what sensor you use; refer our notes or previous assignments for help around this.

To implement the actual feature, note that the sensor only needs to be checked when the **door is closing.**



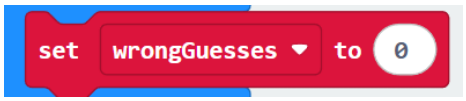
The function that is used to **flash the leds** will have a loop in it; this is a natural place to poll the sensor. *Once per loop (once per LED flash),* check if any object is detected.

If an object is detected, you need to reverse the servo and print a message to the Serial Monitor saying an obstacle was detected. This can be easily done in **ledFlash.** The one difficulty may be the block after **ledFlash**, which sets the **doorState** to **closed**.

- If an obstacle was detected, this variable should end holding "open"
- Since this line of code isn't in ledFlash, when we leave the function it may not be easy to determine if an obstacle was seen or not
    - One workaround would be to have two separate ledFlash type functions; one for opening and one for closing. That way, ALL the code for closing, including obstacle detection, could be in the same function
    - Alternatively, if an obstacle was detected, we could set **doorState** to **"interrupted"** (or something similar), and then check for that value after the **ledFlash** function had completed.
        - In this case, we'd want to set **doorState** back to **"open"**

Attempt Counter and Lockout Mode:

This feature needs a least one variable, something to track the **number of wrong guesses in a row:**



Whenever the button is pressed, this variable should be modified:

- If the current code is correct, **set wrongGuesses to 0** *(reset the incorrect guess counter)*
- Otherwise, **increase wrongGuesses by one.**

In the event that **wrongGuesses** was increased, its possible that the third mistake in a row may have happened, so you should use an **IF** statement to check for that case. If it is:

- Call some function that flashes the neopixel LEDs red for ____ seconds
- Print “lockout” to the Serial monitor
- After the loop used to flash the LEDs is done, reset **wrongGuesses** back to 0.

The only other change needed is to **disable the pushbutton activity** if wrongGuesses is 3

- The variable gets set back to 0 eventually, so add an **IF** statement to your button detection code to only allow the code to run if **wrongGuesses < 3**